

PROJET ECOTRACK

PLANIFICATION AGILE

Mastère EADL — Filière Développement Logiciel (RNCP 38822)

I. CHOIX DE LA MÉTHODOLOGIE AGILE

✓ Critères RNCP : Ce1.4.1 / A1.4 - Méthodologie spécifique

Le projet ECOTRACK retient la méthodologie Scrum, conformément à l'approche préconisée par le cahier des charges. Le besoin reste partiellement à découvrir : les règles d'optimisation des tournées (TSP/2-opt), les seuils d'alerte des conteneurs, les mécaniques de gamification engageantes pour 15 000 citoyens et les usages terrain des agents (scan, mode hors-ligne) doivent être affinés au contact des utilisateurs. Les sprints courts de deux semaines permettent de livrer un incrément fonctionnel à chaque itération et d'ajuster le produit grâce au feedback des gestionnaires, des agents et des citoyens. Le Product Backlog priorisé en MoSCoW garantit que le cœur de valeur (carte temps réel, ingestion MQTT, tournées optimisées, signalement citoyen) est livré avant les fonctionnalités secondaires. Les revues et rétrospectives sécurisent qualité et cadence sur les 4 mois (16 semaines) du projet, tandis qu'une gestion itérative des risques techniques (tests de charge et de couverture dès les premiers sprints) maîtrise un système IoT temps réel à forte incertitude fonctionnelle.

A. Framework Scrum

Rôles : Product Owner, Scrum Master, Équipe Dev

Événements : Sprint Planning, Daily Stand-up, Sprint Review, Rétrospective

Artefacts : Product Backlog, Sprint Backlog, Incrément

B. Organisation en Sprints

- Durée d'un sprint : 2 semaines
- Cadençage : Kick-off (S1), Sprint 0 (S2-3), Sprints 1-5 puis Soutenance (S15-16)
- Durée totale projet : 4 mois (16 semaines, janvier à avril)

II. BACKLOG PRODUIT

A. Product Backlog (filière Développement) — format « En tant que... je veux... afin de... » / MoSCoW / Story Points

US-001 : En tant que Citoyen, je veux signaler un conteneur plein depuis l'application mobile, en ajoutant une photo, afin d'alerter le service de collecte et gagner des points | MUST | 8 SP

US-002 : En tant que Système/API, je veux ingérer les mesures des capteurs via MQTT (toutes les 15 min, ~500 000 messages/jour) afin de connaître l'état de remplissage des 2 000 conteneurs en continu | MUST | 8 SP

US-003 : En tant que Gestionnaire, je veux visualiser la carte des conteneurs colorés par niveau de remplissage en temps réel afin de superviser l'état du parc et repérer les conteneurs critiques (>90 %) | MUST | 8 SP

US-004 : En tant que Gestionnaire, je veux créer une tournée et lancer son optimisation (TSP + 2-opt, seuil de remplissage paramétrable) afin de minimiser les distances parcourues et les coûts | MUST | 13 SP

US-005 : En tant qu'Agent de collecte, je veux recevoir ma tournée optimisée du jour sur mobile (itinéraire, étapes numérotées, distance, durée) afin de suivre mon parcours sans perte de temps | MUST | 8 SP

US-006 : En tant qu'Agent de collecte, je veux valider une collecte en scannant le QR code du conteneur (volume, GPS, horodatage) afin de tracer précisément chaque passage | MUST | 8 SP

US-007 : En tant qu'Agent de collecte, je veux utiliser l'application en mode hors-ligne et synchroniser mes validations au retour de connexion afin de travailler dans les zones sans réseau | MUST | 8 SP

US-008 : En tant qu'Utilisateur, je veux m'authentifier via JWT, avec MFA pour les gestionnaires et administrateurs, afin de sécuriser l'accès à la plateforme | MUST | 5 SP

US-009 : En tant qu'Administrateur, je veux gérer les rôles et permissions (RBAC à 4 rôles : Citoyen, Agent, Gestionnaire, Admin) afin de cloisonner les accès selon le principe de moindre privilège | MUST | 5 SP

US-010 : En tant que Citoyen, je veux accumuler des points, débloquent des badges (30+) et suivre ma progression afin d'être récompensé pour mon engagement éco-responsable | MUST | 8 SP

US-011 : En tant que Citoyen, je veux participer à des défis collectifs et consulter un classement (leaderboard) afin de me motiver et de comparer mon impact à celui des autres | SHOULD | 5 SP

US-012 : En tant que Gestionnaire, je veux recevoir des alertes lorsqu'un conteneur dépasse un seuil de remplissage afin d'anticiper les débordements (objectif < 2 %) | SHOULD | 5 SP

US-013 : En tant que Gestionnaire, je veux accéder à un dashboard analytics (volumes collectés, taux de remplissage, distances, CO2 évité) afin de piloter le service et mesurer les gains | SHOULD | 8 SP

US-014 : En tant qu'Utilisateur, je veux recevoir des notifications push et e-mail (alertes, affectation de tournée, défis) afin d'être informé sans surveiller l'écran | SHOULD | 5 SP

US-015 : En tant qu'Agent de collecte, je veux signaler une anomalie terrain (conteneur endommagé, accès bloqué) avec commentaire et photo afin d'alerter le gestionnaire | SHOULD | 3 SP

US-016 : En tant que Système/API, je veux exposer une API REST documentée (OpenAPI/Swagger) avec rate limiting (100 req/min/IP) afin de garantir l'intégration et la robustesse de la plateforme | SHOULD | 5 SP

US-017 : En tant que Gestionnaire, je veux générer un rapport mensuel PDF (KPIs, graphiques : collectes, distances, débordements) afin d'alimenter le reporting institutionnel | COULD | 5 SP

US-018 : En tant qu'Administrateur, je veux configurer les zones, seuils, capteurs et canaux de notification afin d'adapter la plateforme à l'évolution du terrain | COULD | 3 SP

III. PLANIFICATION PAR SPRINT

Kick-off (S1) & Sprint 0 (S2-3) : Cadrage & Socle Technique

- User Stories : US-002, US-008, US-016 — Total 18 SP
- Objectif : Constituer l'équipe, dresser le backlog initial, cadrer l'architecture applicative et poser le socle technique (repos GitFlow, services Dockerisés, CI/CD GitHub Actions, environnements dev/staging) ainsi que l'ingestion MQTT initiale
- Livrables (DEV) : backlog priorisé MoSCoW + squelette API REST (FastAPI/Node.js) + conteneurs Docker + pipeline GitHub Actions (build/test/lint) + broker MQTT opérationnel + authentification JWT/MFA + RBAC initial

Sprints 1-2 (S4-7) : Fondations & Modules Core

- User Stories : US-003, US-005, US-006, US-009 — Total 29 SP
- Objectif : Livrer le cœur fonctionnel — supervision temps réel des conteneurs et premier circuit agent
- Livrables (DEV) : carte temps réel des conteneurs (React/TypeScript + Leaflet, WebSocket) + persistance des mesures IoT + RBAC 4 rôles complet + application mobile agent (réception de tournée, scan QR, validation de collecte) + premiers tests automatisés

Sprints 3-4 (S8-11) : Fonctionnalités Avancées

- User Stories : US-001, US-004, US-007, US-010, US-011, US-015 — Total 37 SP
- Objectif : Déployer l'optimisation des tournées et l'engagement citoyen, fiabiliser le terrain
- Livrables (DEV) : moteur d'optimisation TSP + 2-opt (feuilles de route) + application mobile citoyen (signalement + photo) + mode hors-ligne agent avec synchronisation + gamification complète (points, badges 30+, défis, leaderboard) + signalement d'anomalie agent

Sprint 5 (S12-14) : Intégration, Tests & Polish

- User Stories : US-012, US-013, US-014, US-016 — Total 23 SP
- Objectif : Industrialiser, intégrer les briques et atteindre les exigences non fonctionnelles
- Livrables (DEV) : dashboard analytics (volumes, taux de remplissage, distances, CO2 évité) + moteur d'alertes sur seuils + notifications push/e-mail + API REST documentée (Swagger) avec rate limiting 100 req/min/IP + montée de la couverture de tests > 60 % + tests de charge (500 users simultanés, API < 500 ms p95)

Soutenance (S15-16) : Recette, Documentation & Présentation

- User Stories : US-017, US-018 — Total 8 SP
- Objectif : Finaliser, documenter et préparer la présentation finale au jury
- Livrables (DEV) : rapports mensuels PDF + backoffice de configuration (zones, seuils, capteurs, canaux) + DCT (30-35 pages) + README + documentation API + code GitHub + application déployée + démonstration de soutenance

IV. OUTILS DE GESTION

- Outil principal : Jira ou GitHub Projects
- Communication : Slack ou Discord
- Versioning : GitHub avec GitFlow
- CI/CD : GitHub Actions ou GitLab CI
- Documentation : Confluence ou Notion

V. INDICATEURS CLÉS DE PERFORMANCE (KPI)

A. KPIs Techniques (filière Développement)

1. Couverture de tests : $(\text{Lignes testées} / \text{Total}) \times 100$ | Cible $> 60 \%$ | Par sprint
2. Temps de réponse API : Latence des endpoints critiques | Cible < 500 ms (p95) | Temps réel
3. Disponibilité système : Uptime de la plateforme | Cible $\geq 99,5 \%$ | Journalier
4. Performance requêtes SQL : Temps d'exécution | Cible < 200 ms pour 90 % des requêtes | Temps réel
5. Vitesse sprint : Story Points complétés par sprint | Cible 25 ± 3 SP | Par sprint
6. Bugs critiques : Nombre de bugs P1 ouverts | Cible 0 | Journalier
7. Cache hit ratio : $(\text{Requêtes servies par le cache} / \text{Total}) \times 100$ | Cible $> 80 \%$ | Temps réel
8. Temps de chargement page : Affichage initial de l'interface | Cible < 3 s | Par release
9. Charge supportée : Utilisateurs simultanés sans dégradation | Cible ≥ 500 | Par campagne de charge
10. Lead time for changes : Délai commit $\rightarrow$ production | Cible < 2 jours | Par release
11. Fréquence de déploiement : Nombre de déploiements en production | Cible $\geq 1/\text{semaine}$ | Hebdo
12. Taux d'échec des changements : $(\text{Déploiements en échec} / \text{Total}) \times 100$ | Cible $< 15 \%$ | Par release
13. MTTR (rétablissement) : Délai moyen de restauration du service après incident | Cible < 1 h | Par incident

B. KPIs Métier

14. Réduction des distances de collecte : $(\text{KM économisés} / \text{KM avant}) \times 100$ | Cible $\geq 20 \%$ | Mensuel
15. Taux de débordement : $(\text{Conteneurs débordés} / \text{Total}) \times 100$ | Cible $< 2 \%$ | Hebdo
16. Citoyens actifs : Nb d'utilisateurs actifs sur la plateforme | Cible 15 000 | Mensuel
17. Réduction des coûts opérationnels et CO2 : Coûts -15% / émissions -18% | Cible atteinte | Mensuel

VI. GESTION DES RISQUES

Risques Communs (toutes filières) :

- Retard développement fonctionnalité critique | Prob: Moyenne | Impact: Élevé
- Départ membre équipe | Prob: Faible | Impact: Moyen
- Dépassement budget | Prob: Moyenne | Impact: Élevé

Risques Spécifiques (filière Développement) :

- Dette technique accumulée | Mitigation : code reviews systématiques + refactoring planifié
- Volumétrie IoT MQTT mal absorbée (~500k msg/j) | Mitigation : tests de charge dès les Sprints 1-2 + cache + file de messages
- Bugs critiques en production | Mitigation : tests automatisés (>60 %) + environnement de staging + CI/CD

VII. RESSOURCES ET BUDGET

A. Équipe Projet

- Product Owner : 1 personne — 25 % temps
- Scrum Master : 1 personne — 25 % temps
- Développeurs full-stack : 3 personnes — 100 % temps
- Architecte logiciel : 1 personne — 50 % temps

B. Budget Prévisionnel (16 semaines)

- Ressources Humaines : 116 k€
- Infrastructure Cloud : 3 k€
- Outils & Licences : 2 k€
- Formation : 3 k€
- Imprévus (10 %) : 12 k€

TOTAL : 136 k€

VIII. DIAGRAMME DE GANTT

Planning sur 4 mois (16 semaines) suivant les jalons du cahier des charges (jalons en gras) :

- S1 — Kick-off : équipe constituée, backlog initial priorisé | Jalon : lancement projet
- S2-3 — Sprint 0 : architecture applicative, environnements, CI/CD, ingestion MQTT initiale, auth JWT/MFA | Jalon : socle technique livré
- S4-7 — Sprints 1-2 : carte temps réel, RBAC, persistance IoT, app mobile agent (tourn e/scan QR) | Jalon : modules core
- S8-11 — Sprints 3-4 : optimisation TSP/2-opt, app mobile citoyen (signalement+photo), mode offline, gamification | Jalon : fonctionnalit s avanc es
- S12-14 — Sprint 5 : dashboard analytics, alertes, notifications, API document e, tests de charge & couverture > 60 % | Jalon : int gration, tests, polish
- S15-16 — Soutenance : rapports, backoffice config, DCT, d ploiement, d monstration | Jalon : pr sentation finale

IX. SYNTHÈSE & AMÉLIORATION

A. Récapitulatif

- Méthodologie : Scrum / sprints de 2 semaines sur 4 mois (jalons CDC : Kick-off, Sprint 0, Sprints 1-5, Soutenance)
- Backlog : 18 user stories priorisées (MoSCoW)
- KPIs : 17 indicateurs (13 techniques + 4 métier)
- Budget : 136 k€ sur 4 mois (16 semaines)

B. Axes d'Amélioration (3-5 axes)

1. Automatisation des tests end-to-end : intégrer une suite E2E (Playwright) dans la CI afin de fiabiliser les parcours critiques (signalement citoyen, tournée agent, optimisation) à chaque déploiement.
2. Évolution de l'algorithme d'optimisation : enrichir le couple TSP/2-opt par une métaheuristique (recuit simulé ou algorithme génétique) afin de pousser le gain kilométrique au-delà des 20 % visés.
3. Observabilité : déployer un stack de monitoring (Prometheus/Grafana, traces distribuées) afin de réduire le MTTR et d'anticiper les dérives de performance de l'ingestion MQTT (~500k msg/j).